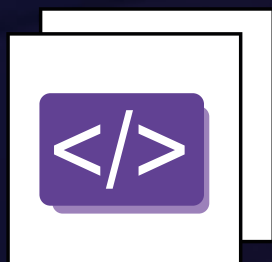




O PODER DO FUTURO

Formação introdutória à tecnologia
para jovens mulheres negras



Introdução ao CSS





INTRODUÇÃO AO CSS

O CSS (*Cascading Style Sheets*, ou Folhas de Estilo e Cascata em português) é uma linguagem de marcação de texto responsável por dar estilo a uma página *web*. Enquanto o HTML é responsável por estruturar o esqueleto de uma página, o CSS dá o estilo e preenche o corpo da página com cores e formatos. Em 1995, Bert Bos e Håkon Wium perceberam que era importante separar responsabilidades e proporcionar estilo à páginas *web*. Decidiram então apresentar a proposta do CSS à W3C, que apoiou a ideia.

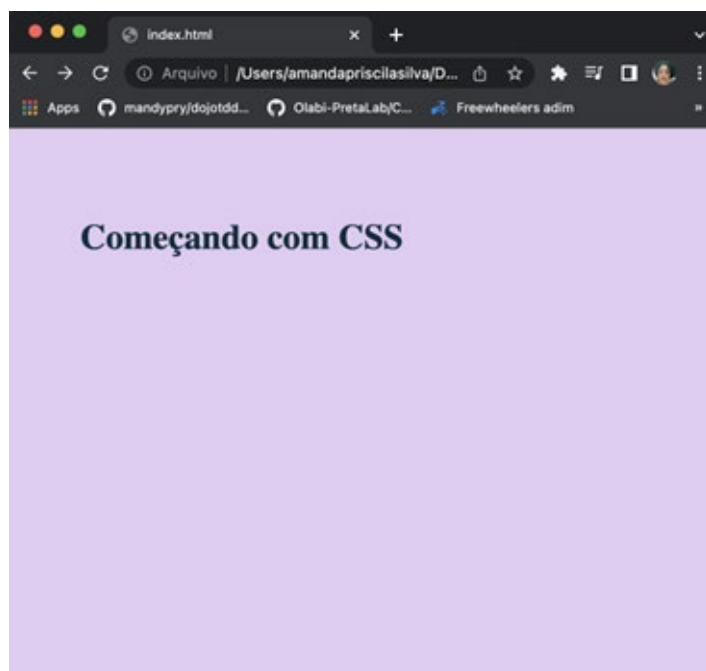
CSS interno, inline e externo

Existem três maneiras de comunicarmos nosso arquivo HTML com o CSS:

Interno: é uma forma de conferirmos estilo ao nosso código no cabeçalho do HTML (ou seja, dentro da nossa *tag* **<head>**) com a *tag* **<style>**. No código a seguir, chamaremos nosso primeiro *style* dentro da *tag* **<head>** (linhas 4 à 12), definiremos a cor de fundo através da *tag* *body* e uma outra cor ao texto dentro da *tag* **<h1>**.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <style>
5        body {
6          background-color: rgba(162, 85, 213, 0.305);
7        }
8        h1 {
9          color: rgb(8, 47, 62);
10         padding: 60px;
11       }
12     </style>
13   </head>
14
15   <body>
16     <h1>Começando com CSS</h1>
17   </body>
18 </html>
```

O resultado em nosso navegador será:

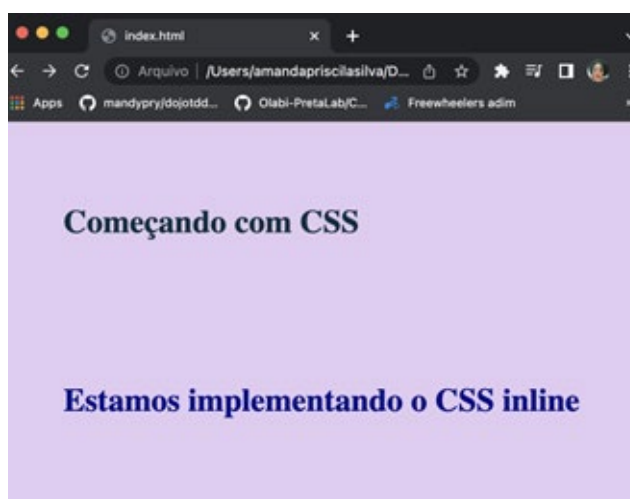


Inline

É possível passar o estilo dentro da própria *tag* através do atributo *style*. Dentro do nosso HTML, vamos inserir outro trecho de código (com a *tag* `<h1>`) e passar uma cor azul com o atributo *style*, para estilizar nosso segundo trecho de texto.

```
15 <body>
16   <h1>Começando com CSS</h1>
17   <h1 style="color: darkblue;"> Estamos implementando o CSS inline</h1>
18 </body>
```

O resultado no nosso navegador será:



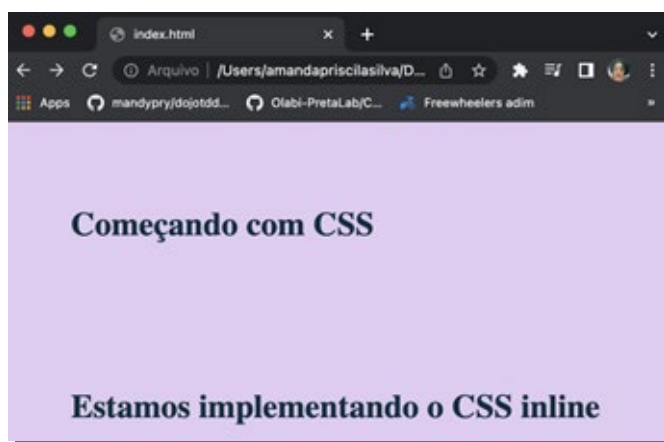
Externo: O CSS externo é a melhor e mais recomendada forma de separar as responsabilidades de cada código pensando nas boas práticas. É preciso criar um arquivo com a extensão **.css** e chamá-lo dentro do arquivo HTML na tag **<head>**, com a tag **<link />**. Dentro do mesmo repositório (pasta) no qual se encontra o nosso arquivo index.html, vamos criar um index.css e passar o código dentro do **<head>** com a tag **<style>**.

```
# style.css U x
pretalab > # style.css > ...
1  body {
2      background-color: rgba(162, 85, 213, 0.305);
3  }
4  h1 {
5      color: rgb(8, 47, 62);
6      padding: 60px;
7  }
```

Dentro do **HTML**, na tag **<head>**, vamos chamar o nosso arquivo com a tag **<link />**.

```
<head>
  <link rel="stylesheet" type="text/css" href="style.css"/>
</head>
```

Onde o atributo **rel** disser que o arquivo importado é responsável pelo estilo da página, o **type** o definirá como um texto em CSS - e o **href** passará o caminho do nosso arquivo CSS. O resultado no navegador será o mesmo.



Comentários no CSS

No arquivo CSS, existe a possibilidade de escrevermos comentários não visíveis ao usuário na tela, que ficarão apenas para o nosso código. Normalmente, usamos pequenos comentários para descrever o que determinado trecho de código faz; até quando estamos aprendendo, podemos deixar comentários que nos ajudem a lembrar o que estamos fazendo.

Para incluir comentários no código, usamos a seguinte sintaxe:

- ***/**** Isto é um comentário ****/*** ;
- Aonde, ***/**** diz para o meu arquivo CSS que iniciei um comentário; quando um ****/*** é encontrado, o comentário acaba;
- Os comentários podem ter muitas linhas ou apenas uma; tudo que for digitado depois da tag ***/**** será interpretado como comentário, inclusive quebra de linha;

Abaixo, temos um exemplo no código de como usar comentários. Observem na linha **1** um comentário de uma linha e, entre as linhas **3**, **4** e **6** à **11**, um único comentário, com várias linhas:

```
1  /* primeiras propriedades: */
2  * {
3      /* define a área de margem nos quatro
4      lados do elemento */
5      margin: 0;
6      /*
7      distância entre
8      o conteúdo de
9      um elemento e
10     suas bordas
11     */
12     padding: 0;
13     /* para configurar uma ou mais das propriedades de contorno */
14     outline: 0;
15 }
```

Sintaxe básica

Agora, vamos entender a sintaxe.

```
.classe{  
  propriedade: valor;  
}
```

Sistema de Cores

O CSS trabalha com um esquema de cores **nominais**, **hexadecimais**, ou em **rgb**:

- Cores nominais são passadas em inglês mesmo no código (por exemplo, **color: red**);
- Cores **hexadecimais** são definidas por um código de seis letras, ou números precedidos do **#** - os dois primeiros representam a intensidade de vermelho; o terceiro e o quarto, a intensidade de verde; e os dois últimos, a intensidade de azul (cores-base de qualquer uma). Por exemplo, o código **#FFFFFF** representa a cor branca;
- As cores rgb são baseadas no padrão *red, green, blue*; as tonalidades de cada cor são representadas por valores entre 0 e 255. O código **rgb(255,255,255)** representa a cor vermelha.

A propriedade *background-color* representa a cor de fundo de um elemento no **CSS**; a propriedade **color** representa a cor da fonte de um elemento no CSS.

Abaixo, o código em HTML ainda chamando o nosso CSS com a tag **<link>**:

```
<!DOCTYPE html>  
<html>  
  <head>  
    <link rel="stylesheet" type="text/css" href="style.css"/>  
  </head>  
  
  <body>  
    <h1>Esquema de cores em CSS</h1>  
    <h1> Temos um corpo roxo e a fonte rosa claro</h1>  
  </body>  
</html>
```


Em seguida, o código CSS, onde definimos a cor roxa para o corpo da nossa página e a cor rosa para o texto:

```
body {  
  background-color: #6959CD;  
}  
h1 {  
  color: #FFB6C1;  
}
```

O resultado na tela será:



Medidas absolutas

Usamos as medidas absolutas pixel, percentagem “em” e “vh” para definir medidas em alguns atributos do CSS:

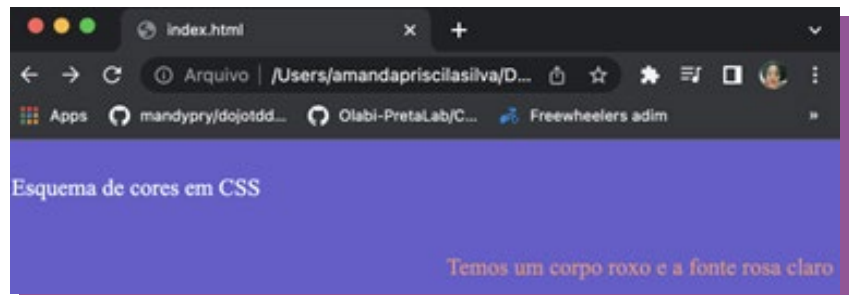
- **Pixels (px):** Relativos ao dispositivo de visualização. Para dispositivos de baixo pontos por polegada, 1px é um pixel de dispositivo (ponto) da tela. Para impressoras e telas de alta resolução, 1px implica em vários pixels de dispositivo;
- **Percentagem (%):** Mede em relação ao comprimento pai. Quando a largura do “pai” é 800px, 50% torna-se 400px;
- **Em:** Unidades de comprimento relativo especificam um comprimento relativo a outra propriedade de comprimento. As unidades de comprimento relativo escalam melhor entre diferentes meios de renderização;
- **Vh:** corresponde a 1/100 da altura da *viewport*. Se a altura do navegador é 900px , 1vh equivale a 9px; analogamente, se a largura da *viewport* é 750px , 1vw equivale a 7.5px.

Alinhamento

É possível alinhar elementos com CSS na horizontal e na vertical, por meio de diversas propriedades. Vamos usar o atributo *text-align* como exemplo para alinhar o título do texto no lado esquerdo e a frase, no direito:

```
#titulo {
  color: azure;
  text-align: left;
}

.frase {
  color: darksalmon;
  text-align: right;
}
```



Fontes

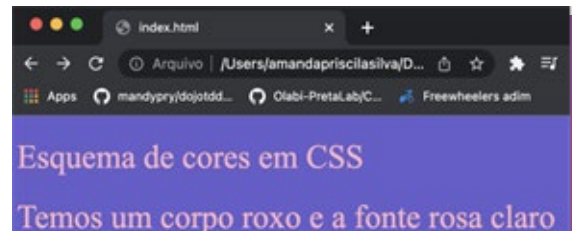
É possível categorizar fontes de diversas formas definindo uma lista a ser usada na ordem; a última fonte da lista deverá ser uma fonte genérica, dentre uma das cinco disponíveis no CSS: *serif*, *sans-serif*, *monospace*, *cursive*, ou *fantasy*.

O atributo *font-family* altera o tipo de fonte via CSS:

```
h1 {
  color: #FFB6C1;
  font-family: 'Times New Roman';
}
```

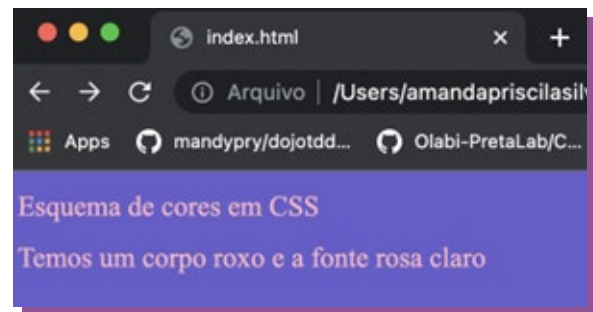
O atributo *font-weight* altera a espessura da fonte com o CSS:

```
h1 {
  color: #FFB6C1;
  font-family: 'Times New Roman';
  font-weight: 100;
}
```



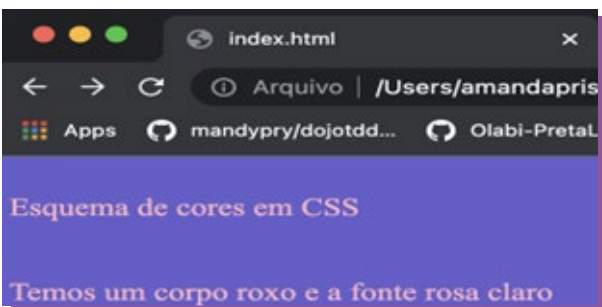
O atributo **font-size** altera o tamanho da fonte com o CSS:

```
h1 {
  color: #FFB6C1;
  font-family: 'Times New Roman';
  font-weight: 100;
  font-size: large;
}
```



O atributo **line-height** é responsável por definir a distância entre as linhas de texto e o CSS:

```
h1 {
  color: #FFB6C1;
  font-family: 'Times New Roman';
  font-weight: 100;
  font-size: large;
  line-height: 50px;
}
```



Identificadores e classes

É possível usar identificadores em *tags* HTML por meio da palavra *id* e chamá-los no CSS desde que precedidos por # (cerquilha); classes são representadas pela palavra *class*, e podem ser chamá-las no css precedidas por . (ponto).

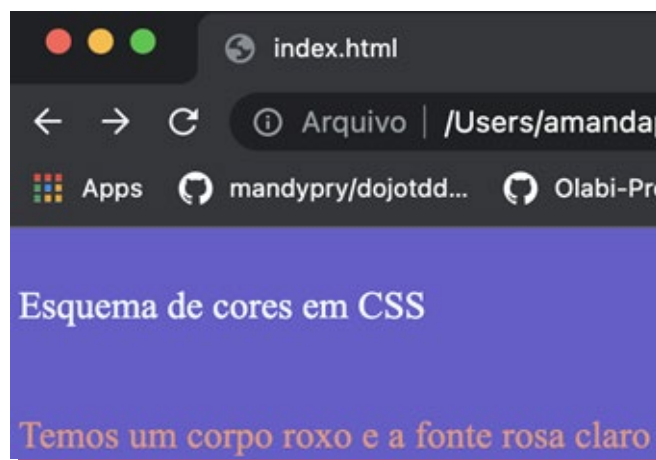
Em nosso HTML, em cada uma das *tags* **<h1>**, passamos respectivamente um **id** e uma **class**:

```
<body>
  <h1 id="titulo">Esquema de cores em CSS</h1>
  <h1 class="frase"> Temos um corpo roxo e a fonte rosa claro</h1>
</body>
```

No CSS, criamos um bloco para cada um:

```
#titulo {  
  color: azure;  
}  
  
.frase {  
  color: darksalmon;  
}
```

O resultado na tela será:



Altura e largura

Usamos as propriedades *width* (largura) e *height* (altura) para definir a dimensão de qualquer elemento. Elas podem assumir valores em comprimento (como px, mm e etc.); percentagem (como 40%); ou nenhum (o padrão). As propriedades *min-width* e *min-height* são usadas para definir largura e altura mínimas do elemento.

As propriedades *max-width* e *max-height* são usadas para definir largura e altura máximas do elemento - quando o valor for definido como "nenhum", não há largura e altura máximas definidas para o elemento.

Bordas a elementos

A propriedade *border*, como o nome sugere, é usada para criar borda no elemento. Podemos alterar cor, largura, ou estilo da borda de um elemento por meio das seguintes propriedades:

- ***border***: maneira abreviada para as quatro bordas;
- ***border-width***: define a espessura;
- ***border-style***: define o estilo;
- ***border-color***: define a cor;
- ***border-top***: propriedades da borda superior;
- ***border-right***: propriedades da borda da direita;
- ***border-bottom***: propriedades da borda inferior;
- ***border-left***: propriedades da borda da esquerda;

Em nosso HTML, atribuímos uma classe e uma *id* a duas *tags* **<h1>**:

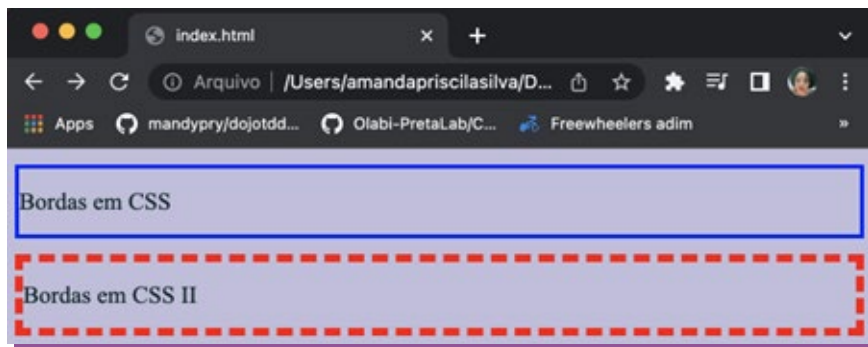
```
<body>
  <h1 id="borda">Bordas em CSS</h1>
  <h1 class="frase">Bordas em CSS II</h1>
</body>
```

Em nosso CSS, usamos algumas das propriedades de borda:

```
#borda {
  color: □ rgb(4, 37, 37);
  border-width: medium;
  border-style: solid;
  border-color: □ #00f;
}

.frase {
  color: □ rgb(4, 37, 37);
  border-width: 6px;
  border-style: dashed;
  border-color: □ #f00;
}
```

O resultado no navegador será:



Espaçamento

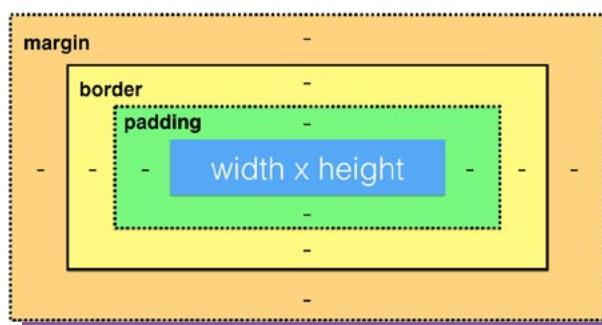
A propriedade *padding* define a distância entre o conteúdo de um elemento e suas bordas:

- ***padding***: define o espaçamento nos 4 lados;
- ***padding-top***: define o espaçamento superior;
- ***padding-right***: define o espaçamento à direita;
- ***padding-bottom***: define o espaçamento inferior;
- ***padding-left***: define o espaçamento à esquerda.

Mais abaixo abaixo, a parte azul define a largura e a altura, a dimensão do elemento. Vamos focar na parte verde que é o preenchimento da parte azul.

Margem

A propriedade *border*, como o nome sugere, representa um espaço em torno de um elemento que o separa de outros. Podemos definir o valor da margem em comprimento, percentagem, automático, ou herança.

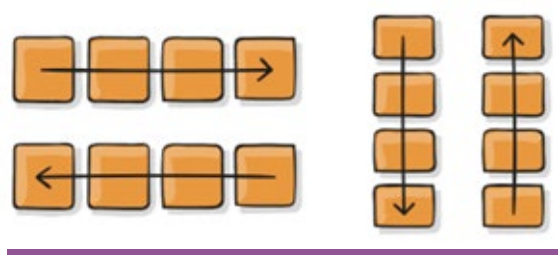


O primeiro passo para escrever códigos em HTML é definir qual ferramenta usar.

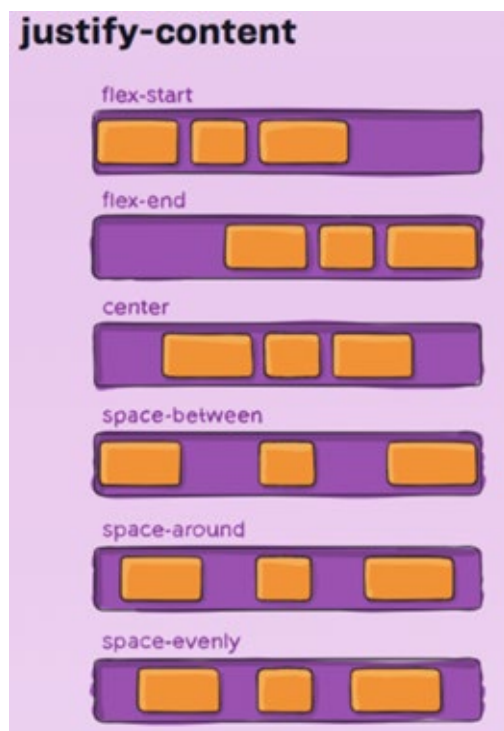
Direção flexível (display, float, clear, clear fix)

É possível definir como ícones ficarão dentro de um container (caixa em torno de vários itens):

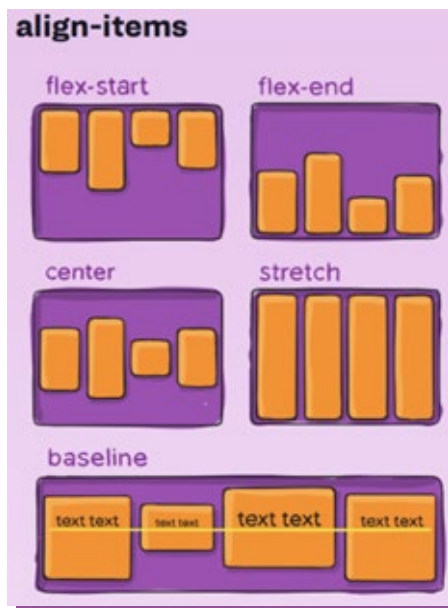
- **flex-direction**: Define como os itens são colocados no contêiner flexível definindo o eixo principal e a direção (normal ou invertida);



- **justify-content**: Define o eixo principal e, como consequência, a direção na qual seus *flex-items* serão colocados. Isso também permite alterar a ordem dos itens, algo que costumava exigir a alteração do HTML subjacente (propriedades *flex-start*, *flex-end*, *center*, *space-between*, *space-around*, *space-evenly*);



- **align-items**: define o valor em todos os “filhos” diretos como um grupo dentro de um contêiner (propriedades incluem *flex-start*, *flex-end*, *center*, *stretch* e *baseline*);



- **display**: Define a organização e o tipo de caixa renderizada em uma página *web* de forma organizada (*inline* ou bloco, dependendo do valor fornecido), além de permitir um contexto flexível para todos os seus “filhos” diretos. O display trabalha como uma tabela na qual o conteúdo pode ser disposto em linhas e colunas;
- **float**: Determina se um item será colocado ao longo do lado direito ou esquerdo do contêiner (onde textos e elementos em linha se posicionaram ao seu redor);
- **clear**: Em conjunto com o *float*, a propriedade é usada quando o próximo elemento precisa ficar abaixo e não ao lado (como ficaria se fosse só *float*);
- **clear fix**: Caso algum dos elementos dentro do contêiner seja mais alto que o outro, com a propriedade *clearfix*, conseguimos igualar o espaçamento entre eles;

imagem retirada do w3school

Sem Clearfix

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum...



Com Clearfix

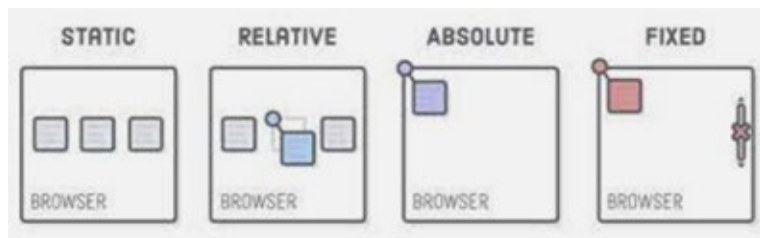
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum...



Posicionamento

Posicionamento é parte crucial na criação de páginas *web*; logo, é importante conhecer e entender propriedades que posicionem elementos dentro da página:

- A propriedade ***position*** recebe os valores ***static***, ***relative***, ***absolute*** e ***fixed*** para posicionar elementos;
- A propriedade ***static*** é o valor padrão de qualquer página HTML que siga o fluxo comum;
- Com ***relative***, o *position* aceita as propriedades *top*, *bottom*, *left*, ou *right* (topo, inferior, esquerda, ou direita). Por meio destas, é possível alterar o posicionamento do elemento;
- A propriedade ***absolute*** permite posicionar qualquer elemento de acordo com o “pai” com *position* diferente de *static*;
- A propriedade ***fixed*** se assemelha à *absolute*, mas referencia a janela do seu navegador (a área que aparece para o usuário independente da barra de rolagem).



Referências:

W3School: [w3school CSS](#)

DevMedia: [devmedia](#)

Developer mozilla: [Developer mozilla CSS](#)

FICHA TÉCNICA

Equipe Olabi: Gabriela Agustini, Silvana Bahia, Aldren Flores, Davi Arloy, Joyce Santos, Amanda Oliveira, Roberta Hércias, Clara Queiroz e Rodrigo Schmitt

Gestão do Projeto: Aldren Flores

Assistente do Projeto: Joyce Santos

Facilitadora: Vera Félix

Comunicação: Raiz Digital

Social media: Raiz Digital e Amanda Oliveira

Coordenadora Técnica: Amanda Silva

Percurso metodológico: Amanda Silva

Conteúdo metodológico das apostilas: Amanda Silva

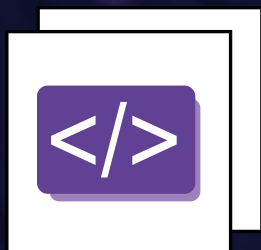
Revisão gramatical: Luciana Moletta

Design: Bruna Martins

Professoras: Amanda Silva, Letícia Furtado, Lisandra Souza, Mônica Santana, Renata Nunes e Simara Conceição

Monitoras: Ana Beatriz dos Santos, Angela Karolina Lopo e Renata Silva

Apoio: Disney



Introdução ao CSS

O PODER DO FUTURO

Formação introdutória à tecnologia
para jovens mulheres negras



Olabi